

E3-TP3 : LANGAGES COMPILÉS -

Compte Rendu

Date : 15 Décembre 2025

1. Introduction

(01) Compilateur C++

Le compilateur C++ utilisé par notre distribution Linux est **g++** (GNU C++ Compiler).

(02) Les deux étapes de la compilation

Pour obtenir un binaire exécutable à partir de sources C++, les deux étapes principales sont :

1. **La compilation** : Traduction du code source (.cpp) en code objet (.o ou .obj). Le compilateur vérifie la syntaxe et génère le code machine pour chaque fichier indépendamment.
2. **L'édition de liens (Linking)** : Assemblage des différents fichiers objets et des bibliothèques nécessaires pour produire un seul fichier exécutable final.

1.1 Compilation directe d'une application console (1 fichier)

(03) Création du fichier source

Le fichier main_act_1.cpp a été créé dans le répertoire tp3/act1 avec le contenu suivant :

```
#include <iostream>
using namespace std;

int main() {
    cout << "Activité 1" << endl;
    cout << "Code C++ compilé avec g++" << endl;
    return 0;
}
```

(04) Compilation directe par défaut

Commande :

```
g++ main_act_1.cpp
```

Résultat :

Cette commande génère par défaut un fichier binaire nommé a.out.

Exécution :

./a.out

(05) Compilation avec nommage de l'exécutable

Pour définir un nom spécifique pour l'exécutable, on utilise l'option -o.

Commande :

```
g++ main_act_1.cpp -o app_act1
```

Exécution :

```
./app_act1
```

1.2 Variable d'environnement « ? »

(06) Vérification du code de retour

La variable d'environnement \$? stocke la valeur de retour de la dernière commande exécutée. Comme notre programme se termine par return 0;, la valeur doit être 0 (indiquant un succès).

Commande :

```
echo $?
```

Résultat observé :

```
0
```

2. Activité 2 : Compilation Séparée

2.1 Compilation séparée (Plusieurs fichiers sources)

(07) Création des fichiers

Les fichiers main.cpp, function.h et function.cpp ont été créés dans tp3/act2.

(08) Compilation en fichiers objets (.o)

La commande pour compiler un fichier source en fichier objet sans effectuer l'édition de liens utilise l'option **-c**.

Syntaxe générique :

```
g++ -c fichier.cpp (produit fichier.o)
```

Commandes effectuées :

```
g++ -c main.cpp
```

```
g++ -c function.cpp
```

Résultat : Création de main.o et function.o.

(09) Édition de liens et exécution

Pour lier les fichiers objets et créer l'exécutable final :

Commande :

```
g++ main.o function.o -o prog
```

Exécution du binaire :

```
./prog
```

Sortie : Activité 2

2.2 Automatisation avec Make

(10) Création du Makefile de base

Pour automatiser la compilation, nous avons créé un fichier nommé Makefile dans le répertoire courant (tp3/act2). Ce fichier contient les règles de compilation et les dépendances.

Contenu du fichier Makefile :

```
# Makefile
```

```
all: hello
```

```
hello: main.cpp function.cpp
```

```
    g++ -O2 -Wall -g -c main.cpp function.cpp
```

```
    g++ -o hello main.o function.o
```

(Note importante : Les espaces devant les commandes g++ sont impérativement des tabulations)

(11) Exécution de make et test

Nous avons lancé la commande make pour exécuter les règles définies précédemment.

Commande et Sortie :

```
leo@pc-e002-07:~/Bureau/tp3/act2$ make
g++ -O2 -Wall -g -c main.cpp function.cpp
g++ -o hello main.o function.o
```

Le processus s'est déroulé sans erreur, générant l'exécutable hello.

Test du résultat :

```
./hello
```

Sortie attendue : Activité 2

2.3 Makefile Portable

(12) Complétion du Makefile avec variables

Pour améliorer la portabilité, nous avons modifié le Makefile pour utiliser des variables. Cela facilite la maintenance (changement de compilateur, options, liste de fichiers).

Fichier Makefile modifié :

```
# Nom du programme
PROG = hello

# Liste des fichiers sources (.cpp) séparés d'un espace
SRCS = main.cpp function.cpp

# Conversion automatique des fichiers sources (.cpp) en fichiers objets (.o)
OBJECTS = $(SRCS:.cpp=.o)

# Options préprocesseur
CPPFLAGS =

# Options compilateur
```

```
CXXFLAGS = -O2 -Wall -g
```

```
# Options éditeur de lien
```

```
LDFLAGS =
```

```
# Compilation
```

```
all: $(PROG)
```

```
$(PROG): $(SRCS)
```

```
$(CXX) $(CPPFLAGS) $(CXXFLAGS) -c $(SRCS)
```

```
$(CXX) $(LDFLAGS) -o $(PROG) $(OBJECTS)
```

Exécution de la commande make :

```
leo@pc-e002-07:~/Bureau/tp3/act2$ make
```

```
g++ -O2 -Wall -g -c main.cpp function.cpp
```

```
g++ -o hello main.o function.o
```

Le résultat est identique au Makefile précédent, mais la structure est maintenant plus flexible pour de futurs ajouts.

(13) Ajout de la norme C++11

Nous avons modifié le Makefile en ajoutant l'option `-std=c++11` à la variable `CXXFLAGS` pour forcer l'utilisation de la norme 2011 du C++.

Ligne modifiée dans le Makefile :

```
CXXFLAGS = -O2 -Wall -g -std=c++11
```

Makefile complet mis à jour :

```
# Nom du programme
```

```
PROG = hello
```

```
# Liste des fichiers sources (.cpp) séparés d'un espace
```

```
SRCS = main.cpp function.cpp
```

```
# Conversion automatique des fichiers sources (.cpp) en fichiers objets (.o)
```

```
OBJECTS = $(SRCS:.cpp=.o)
```

```
# Options préprocesseur
```

```
CPPFLAGS =
```

```
# Options compilateur
```

```
CXXFLAGS = -O2 -Wall -g -std=c++11
```

```
# Options éditeur de lien
```

```
LDFLAGS =
```

```
# Compilation
```

```
all: $(PROG)
```

```
$(PROG): $(SRCS)
```

```
$(CXX) $(CPPFLAGS) $(CXXFLAGS) -c $(SRCS)
```

```
$(CXX) $(LDFLAGS) -o $(PROG) $(OBJECTS)
```

(14) Ajout de la règle de nettoyage (clean)

Nous avons ajouté une cible clean au Makefile pour supprimer les fichiers objets (.o) et l'exécutable généré. Cela permet de repartir sur une compilation propre.

Makefile final complet avec la règle clean :

```
# Nom du programme
```

```
PROG = hello
```

```
# Liste des fichiers sources (.cpp) séparés d'un espace
```

```
SRCS = main.cpp function.cpp
```

```
# Conversion automatique des fichiers sources (.cpp) en fichiers objets (.o)
```

```
OBJECTS = $(SRCS:.cpp=.o)
```

```
# Options préprocesseur
```

```
CPPFLAGS =
```

```
# Options compilateur
```

```
CXXFLAGS = -O2 -Wall -g -std=c++11
```

```
# Options éditeur de lien
```

```
LDFLAGS =
```

```
# Compilation
```

```
all: $(PROG)
```

```
$(PROG): $(SRCS)
    $(CXX) $(CPPFLAGS) $(CXXFLAGS) -c $(SRCS)
    $(CXX) $(LDFLAGS) -o $(PROG) $(OBJECTS)
```

```
clean:
    rm -f $(OBJECTS) $(PROG)
```

Test du nettoyage :

L'exécution de la commande `make clean` supprime bien les fichiers `main.o`, `function.o` et `hello`.